



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

Python para Ciencia de Datos

Tema 8 **Concurrencia**

Máster F.P. En IA & Big Data ANALYTICS
Curso 2026-2027

UPV

Resultados de aprendizaje

- Describir el concepto de **conurrencia** y diferenciar entre ejecución simultánea y ejecución concurrente
- Describir qué **mecanismos** ofrece Python para implementar programas concurrentes
- Explicar los conceptos de **hilo** y **proceso**, indicando sus características comunes y sus diferencias
- Diferenciar entre programas que hacen **uso intensivo de entrada/salida** y de **procesador**, y asociar qué tipo de concurrencia es mejor en cada caso
- **Utilizar** los módulos ***threading*** y ***multiprocessing*** para crear programas sencillos que utilicen múltiples hilos y múltiples procesos, respectivamente
- Utilizar el módulo ***queue*** en escenarios simples para crear colas de uso concurrente entre distintos hilos de un programa

Índice



1. **Concurrencia**
2. **Módulo *Threading***
3. **Módulo *Multiprocessing***
4. **Módulo *Queue***

Índice



1. Concurrency

2. Módulo *Threading*

3. Módulo *Multiprocessing*

4. Módulo *Queue*

Concurrencia

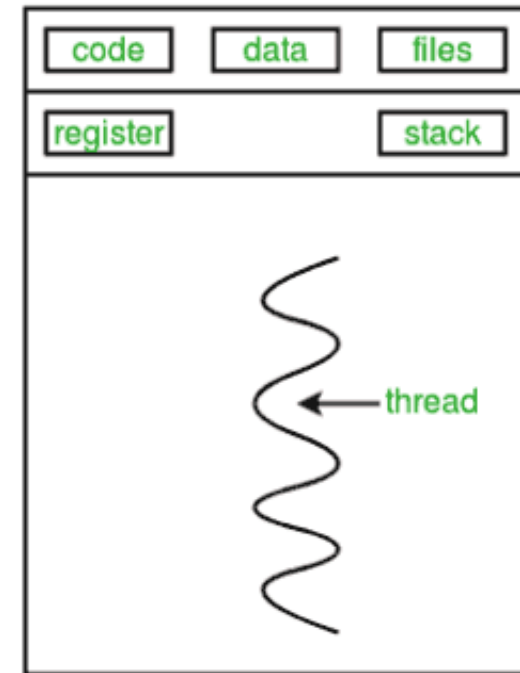
Concepto

- El concepto de **concurrencia** implica la “ejecución simultánea” de distintos “elementos ejecutables” en un sistema informático
 - En realidad, **ejecución concurrente** no siempre significa ejecución simultánea, puesto que suele haber más “elementos ejecutables” que procesadores en el computador
 - Es más preciso pensar que esos elementos **se reparten** los procesadores (dinámicamente), de forma que da la impresión de ejecución simultánea
 - Los “**elementos ejecutables**” pueden ser de distinta naturaleza en función del sistema operativo y del lenguaje de programación utilizado. Los más habituales son los **procesos** y los **hilos**

Concurrencia

Proceso e hilo

- **Proceso** es la abstracción que utiliza el sistema operativo para ejecutar programas
- Un proceso es un **programa en ejecución**, cargado en un **espacio** de memoria independiente
- La **secuencia** de instrucciones que se ejecutan dentro del proceso se denomina **hilo** de ejecución
- El sistema operativo **reparte** de manera dinámica la CPU (o CPUs) entre los procesos activos, pausando o continuando la ejecución de sus hilos
- El sistema debe guardar el **contexto** del hilo cada vez que este abandona la CPU, para poder ponerlo en ejecución más tarde

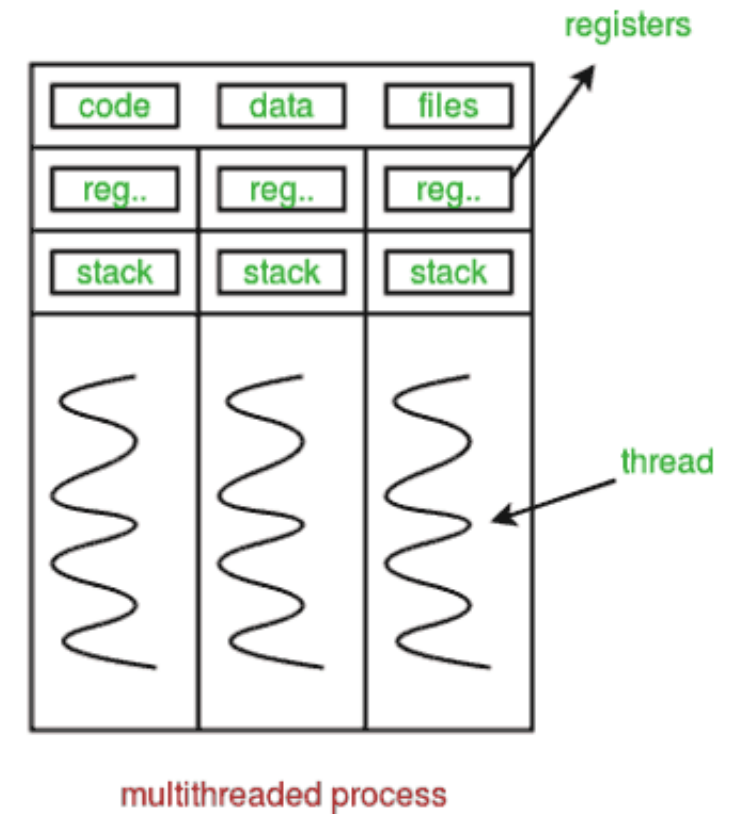


single-threaded process

Concurrencia

Procesos multi-hilo

- En muchos sistemas, es posible ejecutar **múltiples hilos** dentro de un **mismo proceso**
- En ese caso, los hilos **comparten** muchos de los recursos del proceso (memoria, ficheros, etc.)
- Pero cada hilo tiene un **contexto propio** (registros, pila, etc.) para que el sistema pueda pausarlo o reanudarlo de manera independiente
- Los procesos multi-hilo requieren mecanismos de protección ante posibles **condiciones de carrera** al acceder a recursos compartidos
- Esos mecanismos suelen ser algún tipo de herramientas de **sincronización**



Concurrencia

Concurrencia en Python

- Al ejecutar un programa en Python se crea un **proceso** que incluye el **intérprete** de Python y el propio **programa**
- En principio, ese proceso ejecuta un único hilo, que ejecuta el programa principal (“__main__”) y que finaliza cuando terminan sus instrucciones
- Sin embargo, Python proporciona **tres alternativas** para ejecutar distintas **partes del código** del programa de manera concurrente:
 1. **Hilos (*threads*)**, mediante el módulo ***threading***
 2. **Procesos**, mediante el módulo ***multiprocessing***
 3. **Tareas asíncronas**, mediante el módulo ***asyncio***

Concurrencia

Concurrencia en Python

Módulo de Python	Elemento ejecutable	CPUs	Tipo de multitarea	Cambio de ejecución
<i>threading</i>	hilo o <i>thread</i>	una	expulsiva	El sistema operativo reparte la CPU entre los distintos hilos de manera ajena al programa o a Python
<i>multiprocessing</i>	proceso independiente	varias	expulsiva	Varios procesos pueden ejecutarse de manera simultánea en distintas CPUs
<i>asyncio</i>	tarea asíncrona	una	cooperativa	Las tareas abandonan la ejecución voluntariamente

De las tres alternativas, vamos a ver las **dos primeras**

Concurrencia

Programas con uso intensivo de E/S vs CPU

- Normalmente buscamos una solución concurrente a un programa para **acelerar su ejecución**
- Para elegir la mejor alternativa debemos saber si nuestro programa hace más uso de Entrada/Salida (E/S) o de la CPU
- Los **programas de E/S intensiva** centran su actividad en recursos externos (ficheros en disco, red, interacción con usuarios, etc.) cuya respuesta es varios órdenes de magnitud más lenta que la CPU
- Los **programas de CPU intensiva** pasan prácticamente todo el tiempo realizando cálculos (ejecutando instrucciones en la CPU)

Concurrencia

Programas con uso intensivo de E/S vs CPU

- Es decir, tenemos **2 patrones de ejecución** distintos:
 - Los programas de E/S intensiva pasan una parte significativa del tiempo **esperando una respuesta** de los recursos externos, es decir, sin ejecutar instrucciones en la CPU
 - Mientras que los programas de CPU intensiva pasan casi todo el **ejecutando instrucciones** en la CPU
- En consecuencia, si queremos **acelerar la ejecución** del programa:
 - En programas de E/S intensa funcionará bien el uso de **hilos**, ya que mientras un hilo espera un recurso, otro puede ejecutarse
 - En programas de CPU intensa, necesitaremos usar **procesos** independientes, para poder ocupar múltiples CPUs a la vez

Concurrencia

Global Interpreter Lock (GIL)

- En muchos lenguajes de programación, cuando un programa tiene varios hilos, estos pueden ejecutarse **en paralelo** entre las CPUs del computador
- Sin embargo, en Python esto no ha sido posible tradicionalmente, debido al denominado ***Global Interpreter Lock (GIL)***
 - El GIL es un mecanismo de bloqueo que impide que varios hilos puedan ejecutarse en paralelo para un mismo intérprete de Python
 - En un programa multi-hilo, solo uno puede ocupar la CPU en un momento dado
- Nota: Desde Python 3.13 se ha añadido la posibilidad de eliminarlo

Índice



1. Concurrencia

2. Módulo *Threading*

3. Módulo *Multiprocessing*

4. Módulo *Queue*

Módulo *Threading*

Creando hilos

- El módulo ***threading*** habilita la posibilidad de crear **hilos** dentro del programa
- Como hemos visto un hilo es una unidad de ejecución que ejecuta de manera concurrente “una parte” del programa: una **función** del programa (o de cualquier módulo accesible)
- De las muchas posibilidades del módulo *threading*, vamos a explicar la forma más sencilla de ejecutar hilos independientes
- La discusión sobre otros aspectos, como variables locales a un hilo o sincronización entre hilos, quedan fuera del alcance de este tema

Módulo *Threading*

Creando hilos

- Para crear un hilo debemos instanciar un **objeto hilo**, de la clase ***Thread***, cuya **sintaxis** básica es:

```
class threading.Thread(target=None, name=None, args=(), kwargs={})
```

donde:

- target es la función que debe ejecutar el hilo
- name es el nombre que tendrá el hilo
- args es una tupla que incluye los argumentos posicionales que recibirá la función que ejecutará el hilo
- kwargs es un diccionario con los parámetros con nombre que recibirá la función que ejecutará el hilo

Módulo *Threading*

Ejecutando hilos

- Una vez hemos creado uno o varios “objetos hilo”, podemos **iniciar** su ejecución mediante el método `start()`
- Una vez iniciados los hilos, lo habitual es que el programa principal **espere** a que concluya su ejecución. Para ello, se ejecuta el método `join()`
- Podemos saber el **nombre** del hilo se está ejecutando (el nombre que se establece en el argumento `name` al crearlo)

```
hilo.start()
```

```
hilo.join()
```

```
threading.current_thread().name
```


Módulo *Threading*

Ejemplo 1

Programa que
calcula los
cuadrados de los
números de una lista

(1) Versión sin hilos

El identificador del hilo es
MainThread (la función
la ejecuta el “programa
principal”)

```
from threading import *
import os

def calculo_cuadrados(numeros):
    id_hilo = current_thread().name # Identificador hilo
    id_proc = os.getpid()           # Identificador proceso (PID)
    for n in numeros:
        cuadrado = n * n
        print(f"{n}^2 : {cuadrado} | Hilo {id_hilo} | PID {id_proc}")

if __name__ == "__main__":
    lista_numeros = [1, 2, 3, 4, 5, 6, 7, 8]

    calculo_cuadrados(lista_numeros)
```

```
1^2 : 1 | Hilo MainThread | PID 7655
2^2 : 4 | Hilo MainThread | PID 7655
3^2 : 9 | Hilo MainThread | PID 7655
4^2 : 16 | Hilo MainThread | PID 7655
5^2 : 25 | Hilo MainThread | PID 7655
6^2 : 36 | Hilo MainThread | PID 7655
7^2 : 49 | Hilo MainThread | PID 7655
8^2 : 64 | Hilo MainThread | PID 7655
```

Módulo Threading

Ejemplo 1

Programa que
calcula los
cuadrados de los
números de una lista

(2) Versión con hilos

```
1^2 : 1 | Hilo t1 | PID 7655
5^2 : 25 | Hilo t2 | PID 7655
6^2 : 36 | Hilo t2 | PID 7655
7^2 : 49 | Hilo t2 | PID 7655
8^2 : 64 | Hilo t2 | PID 7655
2^2 : 4 | Hilo t1 | PID 7655
3^2 : 9 | Hilo t1 | PID 7655
4^2 : 16 | Hilo t1 | PID 7655
```

```
from threading import *
import os

def calculo_cuadrados(numeros):
    id_hilo = current_thread().name # Identificador hilo
    id_proc = os.getpid()           # Identificador proceso (PID)
    for n in numeros:
        cuadrado = n * n
        print(f"{n}^2 : {cuadrado} | Hilo {id_hilo} | PID {id_proc}")

if __name__ == "__main__":
    lista_numeros = [1, 2, 3, 4, 5, 6, 7, 8]
    medio = len(lista_numeros) // 2
    primera_mitad = lista_numeros[:medio]
    segunda_mitad = lista_numeros[medio:]

    t1 = Thread(target = calculo_cuadrados,
                 name = "t1", args = (primera_mitad,))
    t2 = Thread(target = calculo_cuadrados,
                 name = "t2", args = (segunda_mitad,))

    t1.start()
    t2.start()
    t1.join()
    t2.join()
```

Módulo *Threading*

Ejemplo 2

- Vamos a comparar la **eficiencia** (tiempo de ejecución) de ambas versiones. Para ello:
 - **Ampliaremos** bastante el tamaño de la lista de números
 - Modificaremos el cálculo del cuadrado por **n elevado a n**
- Por comodidad, usaremos una definición de listas denominada “**por comprensión**” (definición funcional), cuya **sintaxis** simplificada es:

```
lista = [expresion for variable in secuencia]
```

- Ejemplos:

```
lista1 = [x for x in range(8)]  
lista2 = [x * 2 for x in lista1]  
lista3 = [0 for i in range(1000)]
```

```
[0, 1, 2, 3, 4, 5, 6, 7]  
[0, 2, 4, 6, 8, 10, 12, 14]  
[0, 0, 0, 0, 0, 0, 0, 0, 0, ...]
```

Módulo *Threading*

Ejemplo 2

(1) Versión sin hilos

Tiempo $\approx$ 8000 ms

```
import os
from time import time
from random import randrange

def calculo_potencias(numeros):
    for n in numeros:
        potencia = n ** n

if __name__ == "__main__":
    # Creamos una lista de 10^8 enteros
    lista_numeros = [randrange(1, 50, 1) for i in range(100000000)]

    t_ini = time()
    calculo_potencias(lista_numeros)
    t_fin = time()

    print(f"Tiempo de ejecución (SIN HILOS): {(t_fin-t_ini)*10**3:8.2f}ms")
```

Tiempo de ejecución (SIN HILOS): 8282.72ms

Módulo *Threading*

Ejemplo 2

(2) Versión con hilos

Tiempo $\approx$ 8000 ms

El coste temporal de la versión con hilos **no ha mejorado** la versión sin hilos. ¿Por qué?

```
import ...

def calculo_potencias(numeros):
    for n in numeros:
        potencia = n ** n

if __name__ == "__main__":
    lista_numeros = [randrange(1, 50, 1) for i in range(100000000)]
    medio = len(lista_numeros) // 2
    primera_mitad = lista_numeros[:medio]
    segunda_mitad = lista_numeros[medio:]

    t1 = Thread(target = calculo_potencias, args = (primera_mitad,))
    t2 = Thread(target = calculo_potencias, args = (segunda_mitad,))

    t_ini = time()
    t1.start(); t2.start()
    t1.join(); t2.join()
    t_fin = time()
    print(f"Tiempo de ejecución (CON HILOS): {(t_fin-t_ini)*10**3:8.2f}ms")
```

Tiempo de ejecución (CON HILOS): 7960.88ms

Módulo *Threading*

Ejemplo 3

- Vamos a plantear otro ejemplo en el que el objetivo del programa es **descargar 10 ficheros** (CSV) de internet
- De nuevo compararemos dos **versiones**: la primera sin concurrencia (un único hilo) y la segunda con hilos concurrentes
- Usaremos el módulo `requests`, que permite descargarse ficheros de una forma cómoda:

```
import requests  
  
req = requests.get(url)  
print(req.text)
```

El método **get()** descarga una **url** en un objeto de la clase `Request`, que tiene entre otros, un atributo `text` con el contenido del sitio descargado



Módulo *Threading*

Ejemplo 3

(1) Versión sin hilos

Tiempo $\approx$ 4500 ms

```
import requests
import time

def descarga_urls(lista_urls, nom_ficheros):
    for i in range(len(lista_urls)):
        req = requests.get(lista_urls[i])
        with open(nom_ficheros[i], "w") as fcsv:
            fcsv.write(req.text)

if __name__ == "__main__":
    lista_sitios = [ "https://valencia.opendatasoft.com/..",
                    "https://public.opendatasoft.com/api/explore..." ] * 5

    lista_nombres_fichero = [f"fichero{i:0>2d}.csv"
                             for i in range(1, len(lista_sitios)+1)]

    t_ini = time.time()
    descarga_urls(lista_sitios, lista_nombres_fichero)
    t_fin = time.time()

    print(f"Tiempo de ejecución (SIN HILOS): {(t_fin-t_ini)*10**3:8.2f}ms")
```

Tiempo de ejecución (SIN HILOS): 4407.52ms

Módulo *Threadi*

Ejemplo 3

(2) Versión con hilos

Tiempo $\approx$ 1800ms

El coste temporal de la versión con hilos es menos de la **mitad** que la versión sin hilos.
¿Por qué?

```
import ...

def descarga_urls(lista_urls, nom_ficheros):
    for i in range(len(lista_urls)):
        r = requests.get(lista_urls[i])
        with open(nom_ficheros[i], "w") as fcsv:
            fcsv.write(r.text)

if __name__ == "__main__":
    lista_sitios = [ "https://valencia.opendatasoft.com/..",
                    "https://public.opendatasoft.com/api/explore..." ] * 5

    lista_nombres_fichero = [f"fichero{i:0>2d}.csv"
                             for i in range(1, len(lista_sitios)+1)]
    mitad = len(lista_sitios) // 2
    t1 = Thread(target=descarga_urls,
                args=(lista_sitios[:mitad], lista_nombres_fichero[:mitad]))
    t2 = Thread(target=descarga_urls,
                args=(lista_sitios[mitad:], lista_nombres_fichero[mitad:]))

    t_ini = time.time()
    t1.start() ; t2.start()
    t1.join() ; t2.join()
    t_fin = time.time()
    print(f"Tiempo de ejecución (CON HILOS): {(t_fin-t_ini)*10**3:8.2f}ms")
```

Tiempo de ejecución (CON HILOS): 1857.80ms

Índice



1. **Concurrencia**
2. **Módulo *Threading***
3. **Módulo *Multiprocessing***
4. **Módulo *Queue***

Módulo *Multiprocessing*

Concurrencia basada en procesos independientes

- El módulo ***multiprocessing*** permite que partes de un mismo programa puedan ser ejecutadas por procesos distintos (planificados por el sistema operativo)
- Cada proceso ejecuta un **intérprete de Python propio**, con lo que eliminamos el problema del *GIL* y se posibilita que los procesos se ejecuten realmente en paralelo (en distintas CPUs)
- La concurrencia basada en procesos **acelera** la ejecución de programas que hacen un **uso intensivo de la CPU** (siempre que sus cálculos puedan realizarse en paralelo)
- Como veremos, la **interfaz** es casi idéntica a la de *threading*

Módulo *Multiprocessing*

Creando procesos

- Para crear un proceso debemos instanciar un **objeto proceso**, de la clase ***Process***, cuya **sintaxis** básica es:

```
class multiprocessing.Process(target=None, name=None, args=(), kwargs={})
```

donde:

- target es la función que debe ejecutar el proceso
- name es el nombre que recibirá el proceso
- args es una tupla que incluye los argumentos posicionales que recibirá la función que ejecutará el proceso
- kwargs es un diccionario con los parámetros con nombre que recibirá la función que ejecutará el proceso

Módulo *Multiprocessing*

Ejecutando procesos

- De manera análoga a los hilos, una vez creados los **objetos** de tipo **proceso**, podemos ponerlos en ejecución mediante el método **start()** y esperar su finalización mediante el método **join()**

```
proceso.start()  
proceso.join()
```

- Podemos recuperar los **atributos** del proceso actual mediante el método:

```
multiprocessing.current_process()
```

Y entre ellos, el nombre (name)

Módulo *Multiprocessing*

Ejemplo

- Vamos a recuperar uno de los **ejemplos** que hemos visto en la sección de hilos, el cálculo de los **potencias** de un listado de números
- Vamos a plantear una implementación totalmente equivalente a la que veíamos con hilos, pero con procesos
- Los tiempos de ejecución de ambas versiones iban alrededor de 8000ms
- Veamos cuál es el coste de la versión con procesos

Módulo *Multiprocessing*

Ejemplo

(3) Versión con procesos

Tiempo $\approx$ 6000 ms

El coste temporal con procesos es un **25% menor** que las otras dos versiones. ¿Por qué?

```
from multiprocessing import *
...

def calculo_potencias(numeros):
    for n in numeros:
        potencia = n ** n

if __name__ == "__main__":
    lista_numeros = [randrange(1, 50, 1) for i in range(10000000)]
    medio = len(lista_numeros) // 2
    primera_mitad = lista_numeros[:medio]
    segunda_mitad = lista_numeros[medio:]

    p1 = Process(target = calculo_potencias, args = (primera_mitad,))
    p2 = Process(target = calculo_potencias, args = (segunda_mitad,))

    t_ini = time()
    p1.start(); p2.start()
    p1.join(); p2.join()
    t_fin = time()
    print(f"Tiempo de ejecución (PROCESOS): {(t_fin-t_ini)*10**3:8.2f}ms")
```

Tiempo de ejecución (PROCESOS): 5880.55ms

Índice



1. **Concurrencia**
2. **Módulo *Threading***
3. **Módulo *Multiprocessing***
4. **Módulo *Queue***

Módulo Queue

El problema de la comunicación en concurrencia

- En programas que utilizan hilos o procesos concurrentes cuya ejecución no es totalmente independiente, es habitual que requieran **compartir información** entre sí
- Una forma habitual de hacerlo es mediante **variables en memoria**, a las que acceden todos los hilos o procesos que necesitan compartir información:
 - En el caso de **hilos** es más sencillo porque de manera natural, todos los hilos de un proceso comparten el mismo espacio de memoria (y, por tanto, las variables globales)
 - En el caso de **procesos**, es necesario habilitar algún mecanismo adicional de compartición de memoria entre ellos

Módulo Queue

El problema de la comunicación en concurrencia

- El acceso concurrente a variables en memoria compartida por parte de distintas entidades de ejecución (hilos, procesos) puede generar problemas:
 - Las **condiciones de carrera** pueden comprometer la consistencia de la información almacenada en variables compartidas si dichos accesos no se realizan garantizando que mientras una entidad modifica la información, no puede haber otras accediendo
- Python incorpora distintos mecanismos para conseguir un **acceso seguro** a variables compartidas (lock, condition, semaphore,...), pero quedan fuera del alcance de este tema
- En su lugar, vamos a ver una solución de alto nivel: la **clase queue**

Módulo Queue

La clase Queue

- Python proporciona un módulo que incorpora una **clase Queue**, que implementa una cola de elementos:
 - Podemos **insertar** (encolar) y **extraer** (desencolar) elementos, en principio en orden FIFO
 - **Múltiples hilos o procesos** pueden acceder a la **misma** cola de manera concurrente para insertar/extraer elementos
 - La implementación interna de la clase garantiza un **acceso seguro**, sin los problemas derivados del uso compartido de la variable
- A continuación, vamos a ver un uso **simplificado** de esta clase

Módulo Queue

La clase Queue

- Para crear una cola, la **sintaxis** es sencilla:

```
cola = queue.Queue(maxsize=0)
```

- Esto instancia el objeto `cola` que es una estructura de datos que puede accederse concurrentemente para añadir y extraer en **orden FIFO** elementos de cualquier tipo
- Si indicamos un tamaño máximo (`maxsize`) entero positivo, ese será el tamaño máximo de elementos que podemos almacenar en la cola en un momento dado. Por defecto, el tamaño es “infinito”

Módulo Queue

La clase Queue

- Aunque no lo vamos a ver, si queremos otros comportamientos tenemos disponibles también estas dos clases:

```
pila = queue.LifoQueue(maxsize=0)  
cola_prio = queue.PriorityQueue(maxsize=0)
```

- Estas clases permiten instanciar, respectivamente, objetos de tipo “pila” (estructura LIFO) o “cola por prioridades” (en la que los elementos se extraen según su prioridad)

Módulo Queue

Insertar y extraer

- Para **insertar** y **extraer** elementos de la cola, tenemos los métodos:

```
cola.put(item, block=True, timeout=None)  
elemento = cola.get(block=True, timeout=None)
```

donde:

- `block`: Si es cierto (por defecto), las llamadas son “bloqueantes” : intentar insertar en una cola “llena” o extraer de una cola “vacía” bloquean al hilo que invoca el método
- `timeout`: Si es `None` (por defecto), el bloqueo es indefinido. En otro caso, este argumento indica el nº de segundos máximo de espera
- Si la llamada no es bloqueante, o bien el bloqueo termina, y no hay posibilidad de insertar o extraer, se genera una excepción

Módulo Queue

Esperar a que las “tareas” finalicen

- De manera muy habitual cuando se trabaja con esta estructura, resulta relevante esperar no solo a que todos los elementos de la cola hayan sido extraídos, sino también **procesados**
- Para eso existen un par de métodos específicos, además de los mencionados anteriormente:

```
cola.join()  
cola.task_done()
```

- En esos casos, el hilo que extraen cada ítem, una vez lo procesa invoca `task_done()`, y el hilo que espera en `join()` se desbloquea cuando se ha invocado `task_done()` para todos los ítems

Módulo Queue

Productor/consumidor

- Cuando usamos colas para intercambiar información, es habitual pensar en **productores** (hilos que insertan) y **consumidores** (hilos que extraen y procesan):

Hilo **productor**

```
for i in items:
    # Crear ítem
    item = ...
    # Insertar ítem:
    cola.put(item)

# No hay más ítems:
cola.put(None)
```

cola



Hilo que **espera***

```
cola.join()
```

Hilo **consumidor**

```
while True:
    # Recuperar ítem
    item = cola.get()
    # Procesar ítem:
    ...
    # Una vez procesado:
    cola.task_done()
    if item == None:
        # No hay más ítems
        break
```

*Por ejemplo, el programa principal

Módulo Queue

Ejemplo: enunciado

- Vamos a plantear un programa similar al que veíamos de descargar y guardar ficheros CSV de internet
- Crearemos 3 hilos:
 - 2 hilos **productores**: se repartirán la tarea de descargar los ficheros de internet e insertar sus contenidos como ítems en una cola
 - 1 hilo **consumidor**: extraerá de la cola cada elemento (contenido) y lo “procesará” (lo guardará como un fichero en disco)
- El programa principal pondrá los hilos en marcha y luego esperará a que los productores terminen, y a que se hayan procesado todos los ítems insertados en la cola

Módulo Queue

Ejemplo: programa principal

```
if __name__ == "__main__":
    lista_sitios = [ "https://valencia.opendatasoft.com/..",
                    "https://public.opendatasoft.com/api/explore..." ] * 5

    mitad = len(lista_sitios) // 2
    cola = Queue()

    print(f"Programa principal: creando hilos")
    t1 = Thread(name='g', target=procesa_fichero, args=(cola,), kwargs={'n_productores' : 2})
    t2 = Thread(name='d1', target=descarga_urls, args=(lista_sitios[:mitad], cola))
    t3 = Thread(name='d2', target=descarga_urls, args=(lista_sitios[mitad:], cola))

    t1.start()                # Iniciando hilo de procesamiento (consumidor)
    t2.start() ; t3.start()   # Iniciando hilos de descarga (productores)

    print(f"Programa principal: Esperando hilos descarga")
    t2.join() ; t3.join()

    print(f"Programa principal: Esperando fin del procesamiento")
    cola.join()               # Esperamos a que todos los ítems de la cola se hayan procesado

    print(f"FIN programa principal")
```

Módulo Queue

Ejemplo: hilos productores

```
def descarga_urls(lista_urls, cola):  
    print(f"Descarga ({current_thread().name}): INICIO")  
  
    # Bucle para procesar la lista de urls:  
    for i in range(len(lista_urls)):  
        req = requests.get(lista_urls[i])  
        print(f"\tDescarga ({current_thread().name}): añadiendo request a la cola")  
        # Insertamos el "request" en la cola:  
        cola.put(req)  
  
    # No hay más urls: insertamos el elemento "vacío" en la cola  
    cola.put(None)  
    print(f"\tDescarga ({current_thread().name}): FIN")
```

Hay **2 hilos** que ejecutan esta función (d1 y d2)

Módulo Queue

Ejemplo: hilo consumidor

```
def procesa_fichero(cola, n_productores = 0):
    print(f"Procesa: INICIO (n_productores: {n_productores})")
    i = 1
    while True:
        req = cola.get()           # Recuperamos el "request" de la cola
        if req == None:           # Si es None, algún productor ha terminado
            n_productores -= 1
            print(f"\tProcesa: None recibido (productores pendientes: {n_productores})")
            if n_productores == 0:
                break             # Si no hay más productores, salimos del bucle
            else:
                cola.task_done()  # Si aun quedan productores, procesamos "None"
                continue          # y vamos a por el siguiente ítem

        nombre_fichero = f"fichero{i:0>2d}.csv"
        print(f"\tProcesa: request {i} recibido: guardando {nombre_fichero}...")
        with open(nombre_fichero, "w") as fcsv:
            fcsv.write(req.text)
        cola.task_done()          # Hemos guardado el fichero (elemento procesado)
        i += 1

    print(f"\tProcesa: No más productores!")
    cola.task_done() # Procesamos el "elemento" pendiente (None) para terminar
    print(f"\tProcesa: FIN")
```

Módulo Queue

Ejemplo: ejecución

Los hilos d1 y d2 van **intercalando** sus ejecuciones conforme se descargan los ficheros, y cada vez que uno de ellos **inserta** uno en la cola, el hilo **procesador** se desbloquea, lo **recupera** y lo **procesa**

Tras cada fichero guardado (o None recibido) el hilo **procesador** invoca `task_done()`

El programa principal **termina** cuando hay tantas invocaciones a `task_done()` como ítems insertados en la cola (10 + 2)

```
Programa principal: creando hilos
Procesa: INICIO (n_productores: 2)
Descarga (d1): INICIO
Descarga (d2): INICIO
Programa principal: Espeando hilos descarga
  Descarga (d2): añadiendo request a la cola
  Procesa: request 1 recibido: guardando fichero01.csv...
  Descarga (d1): añadiendo request a la cola
  Procesa: request 2 recibido: guardando fichero02.csv...
  Descarga (d1): añadiendo request a la cola
  Procesa: request 3 recibido: guardando fichero03.csv...
  Descarga (d2): añadiendo request a la cola
  Procesa: request 4 recibido: guardando fichero04.csv...
  Descarga (d2): añadiendo request a la cola
  Procesa: request 5 recibido: guardando fichero05.csv...
  Descarga (d1): añadiendo request a la cola
  Procesa: request 6 recibido: guardando fichero06.csv...
  Descarga (d1): añadiendo request a la cola
  Procesa: request 7 recibido: guardando fichero07.csv...
  Descarga (d2): añadiendo request a la cola
  Procesa: request 8 recibido: guardando fichero08.csv...
  Descarga (d1): añadiendo request a la cola
  Descarga (d1): FIN
  Procesa: request 9 recibido: guardando fichero09.csv...
  Procesa: None recibido (productores pendientes: 1)
  Descarga (d2): añadiendo request a la cola
  Descarga (d2): FIN
  Procesa: request 10 recibido: guardando fichero10.csv...
Programa principal: Esperando fin del procesamiento
  Procesa: None recibido (productores pendientes: 0)
  Procesa: No más productores!
  Procesa: FIN
FIN programa principal
```